AUTOMATIC TIMETABLE GENERATOR

Unesh Farooqui

Roll no: - 13

Third Year Bachelor of Science

(Computer Science)

Under the Guidance of Prof. Vaishali Desai



LORD UNIVERSAL COLLEGE

Academic Year

2026-27

DECLARATION

I, Mr**. Unesh Farooqui** of **LORDS UNIVERSAL COLLEGE** institute

studying in **T.Y. B.Sc. COMPUTER SCIENCE**, hereby declare that I have completed the field project entitled **AUTOMATIC TIMETABLE GENERATOR** during the academic year 2026-2027.

The report work is original, and the information/data included in the report is true, emerging from the primary and/ secondary data gathered and analyzed as part of this project.

Due credit is extended to the work of Literature/Secondary Survey by endorsing it in the Bibliography as per the prescribed format.

Signature of the Student with Date

Name of Student

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me in the successful completion of my mini project, "Automatic Timetable Generator."

First and foremost, I thank my project guide, Prof. Vaishali Desai, [Designation], Department of [Department Name], for their valuable guidance, constant encouragement, and constructive feedback throughout the development of this project.

I am also grateful to [HOD's Name], Head of the Department of [Department Name], and to our Principal, [Principal's Name], for providing the facilities and a supportive environment to carry out this work.

I extend my thanks to all the faculty members and staff of Lord Universal College for their help and support, and to my friends and classmates for their cooperation and useful suggestions during the project.

Finally, I thank my family for their patience, motivation, and unwavering support, without which this work would not have been possible.

Unesh Farooqui
Roll Number : 13
TYCS 2026-2027

ABSTRACT

Timetable scheduling is one of the most important and time-consuming administrative tasks in any educational institution. Traditionally, timetables are prepared manually by staff members, which requires considerable effort and often leads to problems such as overlapping lectures, teachers assigned to multiple classes at the same time, clashes in classroom or laboratory allocation, and uneven distribution of subjects across the week. Any change in faculty availability or course structure forces the entire process to be repeated, which makes manual scheduling inefficient and error-prone.

This mini project, the **Automatic Timetable Generator**, is designed to overcome these limitations by automating the creation of timetables. The system accepts the required inputs, including the list of subjects, faculty members, classes or divisions, classrooms and laboratories, number of working days, available time slots, and the weekly lecture count for each subject. Using these inputs, it generates a complete, conflict-free timetable automatically within a few seconds.

The system works on a constraint-based scheduling approach. Hard constraints must never be violated, such as a teacher being allotted to only one class at a time, a classroom hosting only one lecture in a given slot, and every subject receiving its required number of lectures per week. Soft constraints improve the quality of the timetable, such as spreading the lectures of each assignment against these constraints, and backtracks or reassigns whenever a conflict is found, until a valid timetable is produced.

The main advantages of the proposed system are a significant reduction in time and manual effort, elimination of scheduling conflicts, optimal use of resources such as classrooms and faculty, and flexibility in adapting to changes. The system also improves accuracy and consistency, since the same rules are applied uniformly to every schedule it creates.

In conclusion, the Automatic Timetable Generator provides an efficient, reliable, and scalable solution to the problem of academic scheduling. It reduces the burden on administrative staff and ensures that students and teachers receive a well-organized timetable. In the future, the system can be extended with features such as exporting timetables to PDF or Excel, handling substitute teachers, supporting multiple departments, and applying advanced optimization techniques such as genetic algorithms to further improve timetable quality.

TABLE OF CONTENTS

CHAPTER 1: PROBLEM IDENTIFICATION & FEASIBILITY STUDY 7

1.1 Problem Identification 7

1.2 Problem Justification and Scope Definition 8

1.3 Stakeholder Identification 9

1.4 Feasibility Analysis 11

1.5 Requirement Engineering & Analysis 13

CHAPTER 2: SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) PLANNING 18

2.1 Selection of SDLC Model 18

2.2 Work Breakdown Structure (WBS) 19

2.3 Project Timeline (Gantt Chart Data) 21

2.4 Resource Requirements 22

CHAPTER 3: SYSTEM DESIGN AND DIAGRAMS 24

CHAPTER 4: SYSTEM ARCHITECTURE DESIGN 27

CHAPTER 5: IMPLEMENTATION 33

5.1 Frontend (HTML, CSS, JavaScript) 33

5.2 Backend (Node.js, Express, MongoDB) 44

CHAPTER 6: SOFTWARE TESTING REPORT 56

6.1 Introduction 56

6.2 Test Environment 56

6.3 Integration Testing 57

6.4 System Testing 59

CHAPTER 7: CONCLUSION 60

CHAPTER 8: FUTURE SCOPE 61

REFERENCES 62

LIST OF FIGURES

Fig. 1 Use Case Diagram

Fig. 2 Class Diagram

Fig. 3 Entity Relationship (ER) Diagram

Fig. 4 Frontend Architecture

Fig. 5 Backend Architecture

Fig. 6 Database Schema Design

Fig. 7 Login Page

Fig. 8 Login Page Showing an Invalid Credentials Error

Fig. 9 Dashboard

Fig. 10 Subjects Page

Fig. 11 Add / Edit Record Dialog

Fig. 12 Teachers Page

Fig. 13 Classrooms Page

Fig. 14 Divisions Page

Fig. 15 Time Slots Page

Fig. 16 Generate Timetable Page

Fig. 17 Generated Timetable (View Timetable Page)

Fig. 18 Settings Page

Fig. 19 Frontend Folder Structure

Fig. 20 Backend Server Running and Connected to MongoDB

Fig. 21 API Root Route Responding in the Browser

Fig. 22 Output of npm test

Fig. 23 Successful Login with the JWT Token (IT-01)

Fig. 24 Login with a Wrong Password (IT-02)

Fig. 25 401 Response When No Token Is Sent (IT-03)

LIST OF TABLES

Table 1 Stakeholder Identification
Table 2 Technical Feasibility Technologies
Table 3 Economic Feasibility Cost Estimation
Table 4 Requirement Prioritization
Table 5 Human Resources Allocation
Table 6 Hardware Resources Specifications
Table 7 Estimated Time Allocation per Phase
Table 8 Software Resources
Table 9 Technology Stack Layers
Table 10 Proposed API Endpoints
Table 11 Conflict Types and System Actions
Table 12 Testing Strategy Matrix
Table 13 Sample Test Cases
Table 14 Frontend Pages and Their Main Functions
Table 15 Frontend Files and Their Purpose
Table 16 API Endpoints of the Backend
Table 17 Collections of the MongoDB Database
Table 18 Scheduling Rules and How They Are Enforced
Table 19 Test Environment
Table 20 Integration Points
Table 21 Integration Test Cases
Table 22 System Test Cases

CHAPTER 1

Problem Identification & Feasibility Study

1.1. Problem Identification

Title of the Problem:

Automatic Timetable Generator

Problem Statement:

Preparing an academic timetable manually is a time-consuming and error-prone task. Educational institutions have to schedule classes for multiple courses, divisions, faculty members, classrooms, and time slots while satisfying several constraints. A small change in one class can require changes in many other entries, which makes manual timetable preparation difficult.

In a manual process, timetable coordinators commonly use paper schedules or spreadsheets. This can lead to faculty clashes, room clashes, overlapping lectures, uneven distribution of subjects, unused classrooms, and difficulty in updating the timetable when a teacher, room, subject, or class timing changes.

Therefore, there is a need for an automated system that accepts academic information and constraints, processes them using scheduling logic, and generates an optimized timetable with minimum or no conflicts. The proposed Automatic Timetable Generator is designed to reduce manual effort and provide a clear timetable that can be viewed, modified, and exported.

Need for the Project: The project is needed because it:

- Reduces the time required to prepare academic timetables.
- Minimizes faculty, classroom, division, and time-slot conflicts.
- Automatically considers subjects, teachers, rooms, and available periods.
- Reduces human errors during timetable preparation.
- Makes timetable modification easier when requirements change.
- Improves classroom and faculty utilization.
- Provides a centralized timetable for administrators and academic staff.
- Allows generated timetables to be displayed and exported for practical use.

1.2. Problem Justification and Scope Definition

Justification

Academic scheduling is a constraint-based problem because several resources have to be allocated at the same time without violating institutional rules. Manual scheduling becomes increasingly difficult as the number of classes, teachers, rooms, and periods increases.

The Automatic Timetable Generator provides a systematic solution by storing academic data in a structured form and applying scheduling rules to generate a timetable. The system can check hard constraints, such as preventing a teacher or classroom from being assigned to two classes at the same time, while also considering soft constraints, such as balancing workloads and avoiding undesirable periods where possible.

Scope of the Project

In Scope

- Administrator registration/login and secure access.
- Management of departments, courses, divisions, subjects, faculty, classrooms, and time slots.
- Configuration of working days and lecture periods.
- Definition of subject requirements such as weekly lecture hours.
- Faculty availability management.
- Classroom availability and capacity management.
- Automatic timetable generation.
- Conflict detection for faculty, classroom, division, and period.
- Timetable viewing by administrator, faculty, and students.
- Manual adjustment of generated timetable entries.
- Timetable regeneration after changes in input data.
- Export/print-ready timetable reports.

Out of Scope

- Automatic attendance management.
- Student examination scheduling.
- Payroll or salary management.
- Learning management and course-content delivery.
- Real-time biometric integration.
- Campus transportation management.

- Full mobile application in the initial version.
- Advanced AI-based prediction of student performance.

1.3. Stakeholder Identification

Table 1: Stakeholder Identification

Stakeholder	Role
Administrator / Timetable Coordinator	Enters academic data, defines constraints, generates and publishes timetables.
Faculty	Views assigned lectures and reports scheduling conflicts or availability issues.
Students	Views the timetable for their class/division.
Department Head	Reviews timetable quality and validates departmental scheduling.
System Administrator	Manages system access, database, backups, and technical configuration.

1.4. Feasibility Analysis

A. Technical Feasibility

The proposed system is technically feasible because it can be developed using commonly available web-development technologies. The scheduling engine can be implemented using rule-based algorithms such as constraint satisfaction, backtracking, greedy scheduling, or a hybrid approach.

Table 2: Technical Feasibility Technologies

Technology	Purpose
Frontend: React.js / HTML5 / CSS3 / JavaScript	User interface and timetable visualization
Backend: Node.js with Express.js	Business logic and API services
Database: MongoDB	Storage of users, subjects, faculty, rooms, periods, and timetable records
Scheduling Engine: Python or JavaScript	Constraint processing and timetable generation
Authentication: JWT	Secure user authentication and authorization
API Testing: Postman	Testing backend APIs
Version Control: Git & GitHub	Source code management

B. Economic Feasibility

The project is economically feasible because the development can use free and open-source tools. The prototype does not require commercial scheduling software. Development can be performed on an existing personal computer with an internet connection.

Table 3: Economic Feasibility Cost Estimation

Item	Estimated Cost
Development Software	Free / Open Source
Programming Languages & Frameworks	Free / Open Source
MongoDB Community Edition	Free
GitHub Account	Free
Testing Environment	Personal Computer
Internet Connection	Existing connection
Development Labour	Self-developed
Initial Development Cost	₹0 (prototype)

If the application is later deployed commercially, additional costs may include cloud hosting, domain registration, SSL certificates, database hosting, backups, and maintenance. These costs are not necessary for the initial academic prototype.

C. Operational Feasibility

The system is operationally feasible because it is intended to simplify, rather than replace, the timetable coordinator's decision-making process. The administrator enters the required academic information and constraints, generates a timetable, reviews the result, and publishes it.

Considerations:

1. User Acceptance

The interface should be simple enough for timetable coordinators and department staff. Users can enter data through forms and generate a timetable with a few steps. A visual grid makes conflicts and free periods easier to identify.

2. Training Requirements

Only basic computer knowledge is required. Users need to understand their institution's subjects, faculty, rooms, working days, and scheduling rules. A short demonstration is sufficient for most administrative users.

3. Maintenance Requirements

The system requires periodic updates when academic structures change. New subjects, faculty, classrooms, divisions, working days, or scheduling rules can be added without redesigning the entire system.

4. Organizational Support

The system can become part of the institution's academic planning process. Department heads can review generated timetables, faculty can check schedules, and students can access the published timetable.

1.5. Requirement Engineering

Functional Requirements

1. User Registration: The system should allow authorized users to create an account.

2. User Login: The system should authenticate users and provide access according to their role.

3. Department Management: The administrator should be able to add, edit, and remove departments.

4. Course/Division Management: The administrator should manage courses, semesters, and divisions.

5. Subject Management: The administrator should define subjects and required weekly periods.

6. Faculty Management: The administrator should store faculty details and teaching assignments.

7. Classroom Management: The administrator should define rooms, capacity, and room availability.

8. Time Slot Management: The administrator should configure working days and lecture periods.

9. Faculty Availability: The system should record periods when a faculty member is unavailable.

10. Constraint Management: The administrator should define hard and soft scheduling rules.

11. Automatic Timetable Generation: The system should generate a timetable based on the configured data and constraints.

12. Conflict Detection: The system should detect faculty, room, division, and time-slot clashes.

13. Timetable Editing: Authorized users should be able to manually adjust generated entries.

14. Timetable Publishing: The administrator should publish an approved timetable.

15. Timetable Viewing: Faculty and students should be able to view the relevant timetable.

16. Report Generation: The system should provide print-ready/exportable timetable reports.

Non-Functional Requirements

- **Performance:** Timetable generation should complete within a reasonable time for the configured institution size.
- **Security:** Only authorized users should modify academic and timetable data.
- **Reliability:** Repeated generation with the same inputs and rules should produce valid schedules.
- **Usability:** The interface should be simple, readable, and easy to navigate.
- **Scalability:** The system should support multiple departments, courses, divisions, faculty members, rooms, and periods.
- **Availability:** Published timetables should be accessible whenever users require them.
- **Maintainability:** Scheduling rules and modules should be separated so that future changes are easier to implement.

Use Case Analysis

Actors:

- Administrator / Timetable Coordinator
- Department Head
- Faculty
- Student
- System Administrator

Major Use Cases:

- Register Account
- Login
- Manage Departments
- Manage Courses and Divisions
- Manage Subjects
- Manage Faculty
- Manage Classrooms
- Configure Time Slots
- Define Faculty Availability
- Define Scheduling Constraints
- Generate Timetable
- Check Conflicts
- Edit Timetable
- Approve/Publish Timetable
- View Timetable
- Export/Print Timetable
- Manage User Account

1.6. Requirement Prioritization

Table 4: Requirement Prioritization

Requirement	**Priority**
User Registration/Login	Must Have
Department/Course/Division Management	Must Have
Subject Management	Must Have
Faculty Management	Must Have
Classroom Management	Must Have
Time Slot Management	Must Have
Automatic Timetable Generation	Must Have
Conflict Detection	Must Have
Timetable Viewing	Must Have
Constraint Management	Should Have
Timetable Export/Print	Should Have

Requirement	Priority
Email/Notification Alerts	Could Have
Multiple Timetable Optimization Strategies	Could Have
Mobile Application	Won't Have (Initial Version)
Attendance/Exam Management	Won't Have (Initial Version)

1.7. Constraints

- **Time Constraint:** The project must be completed within the academic project schedule.
- **Technology Constraint:** The prototype will use selected web technologies and a database.
- **Budget Constraint:** The initial version will use free/open-source software.
- **Data Constraint:** The quality of the generated timetable depends on the correctness of the input data.
- **Scheduling Constraint:** Some institutional rules may be too complex for the initial version and may require future enhancements.
- **Scalability Constraint:** Very large institutions may require optimization of the scheduling algorithm and infrastructure.

Assumptions

- The administrator provides correct subject, faculty, room, division, and period information.
- Faculty availability information is updated when schedules change.
- Rooms listed in the system are available for academic use.
- Users have basic computer knowledge.
- The institution follows a defined working-day and period structure.
- Generated timetables are reviewed by an authorized person before publication.

CHAPTER 2

Software Development Life Cycle (SDLC) Planning

2.1. Selection of SDLC Model

The Agile Software Development Life Cycle (SDLC) Model has been selected for developing the Automatic Timetable Generator. Agile is suitable because timetable requirements can change during development. New constraints, screen requirements, optimization rules, and reporting needs can be added incrementally.

The project can be divided into short iterations called sprints. Early sprints can focus on authentication and master data, followed by timetable generation, conflict detection, dashboard visualization, testing, and deployment. Feedback from users can be incorporated after each sprint.

2.1.1 Key Rationale for Selecting the Agile Model:

- Supports iterative delivery of core modules (e.g., authentication, data entry, scheduling engine).
- Accommodates evolving institutional constraints, faculty preferences, and scheduling logic.
- Facilitates continuous testing, conflict validation, and algorithm refinement across iterations.
- Delivers functional increments early to gather feedback from timetable coordinators.
- Enables incremental optimization of the constraint-satisfaction and backtracking logic.
- Mitigates project risks through phased module validation and stakeholder reviews.

2.2. Work Breakdown Structure (WBS)

The Work Breakdown Structure divides the Automatic Timetable Generator into manageable phases and tasks.

```
AUTOMATIC TIMETABLE GENERATOR
│
├── 1. Project Planning
│    ├── Problem Identification
│    ├── Requirement Gathering
│    ├── Feasibility Study
│    └── Project Planning
│
├── 2. System Design
│    ├── Architecture Design
│    ├── Database Design
│    ├── UI/UX Design
│    ├── Scheduling Algorithm Design
│    └── API Design
│
├── 3. Development
│    ├── User Authentication
│    ├── Department/Course Management
│    ├── Subject Management
│    ├── Faculty Management
│    ├── Classroom Management
│    ├── Time Slot Management
│    ├── Constraint Management
│    ├── Timetable Generation Engine
│    ├── Conflict Detection
│    ├── Timetable Dashboard
│    └── Report/Export Module
│
├── 4. Testing
│    ├── Unit Testing
│    ├── Integration Testing
│    ├── System Testing
│    ├── Algorithm Testing
│    └── User Acceptance Testing
│
├── 5. Deployment
│    ├── Application Configuration
│    ├── Database Configuration
│    └── Final Release
│
└── 6. Maintenance
     ├── Bug Fixes
     ├── Rule Updates
     ├── Performance Improvements
     └── Feature Enhancements
```

2.3. Project Timeline (Gantt Chart)

Activity	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12
Problem Identification & Requirement Engineering	■	■										
SDLC Planning		■	■									
UML Modeling			■	■								
System Architecture & Database Design				■	■							
Core Feature Implementation						■	■	■				
Scheduling Algorithm & Conflict Detection							■	■	■			
Testing & Bug Fixing									■	■		
Deployment & Security Testing										■		
Documentation & Final Submission											■	■

2.4. Resource Planning

1. Human Resources

Table 5: Human Resources Allocation

Resource	Responsibility
Project Guide	Project supervision and academic guidance
Student Developer	Requirement analysis, design, coding, testing, documentation
Tester	Functional, integration, usability, and scheduling validation
End Users	Feedback and validation of generated timetables

2. Hardware Resources

Table 6: Hardware Resources Specifications

Hardware	Specification
Laptop/Desktop	Intel i5 / Ryzen 5 or above
RAM	8 GB minimum
Storage	256 GB SSD or higher
Internet	Stable broadband connection

3. Estimated Time Allocation

Table 7: Estimated Time Allocation per Phase

Phase	Duration
Planning	2 Weeks
Design	2 Weeks
Development	4 Weeks
Testing	2 Weeks
Deployment	1 Week
Documentation	1 Week

4. Software Resources

Table 8: Software Resources

Software	Purpose
Visual Studio Code	Code Editor
Node.js	Backend Runtime
React.js	Frontend Development
Express.js	Backend Framework
MongoDB	Database
Python / JavaScript	Scheduling Algorithm
Git & GitHub	Version Control
Postman	API Testing
Chrome Browser	Application Testing

2.5. Technology Stack

Table 9: Technology Stack Layers

Layer	Technology
Frontend	React.js, HTML5, CSS3, JavaScript
Backend	Node.js, Express.js
Database	MongoDB
Scheduling Engine	Python or JavaScript
Authentication	JWT
API Communication	REST API
Cloud	Render / Vercel or local deployment
Version Control	GitHub

CHAPTER 3

System Modeling using UML

Unified Modeling Language (UML) is used to visualize, specify, and document the structure and behavior of the Automatic Timetable Generator. UML diagrams help explain how administrators, faculty, students, and system components interact before implementation.

The major UML models for this project include the Use Case Diagram, Class Diagram, Entity Relationship (ER) Diagram, Sequence Diagram, Deployment Diagram, and Activity Diagram.

3.1. Use Case Diagram

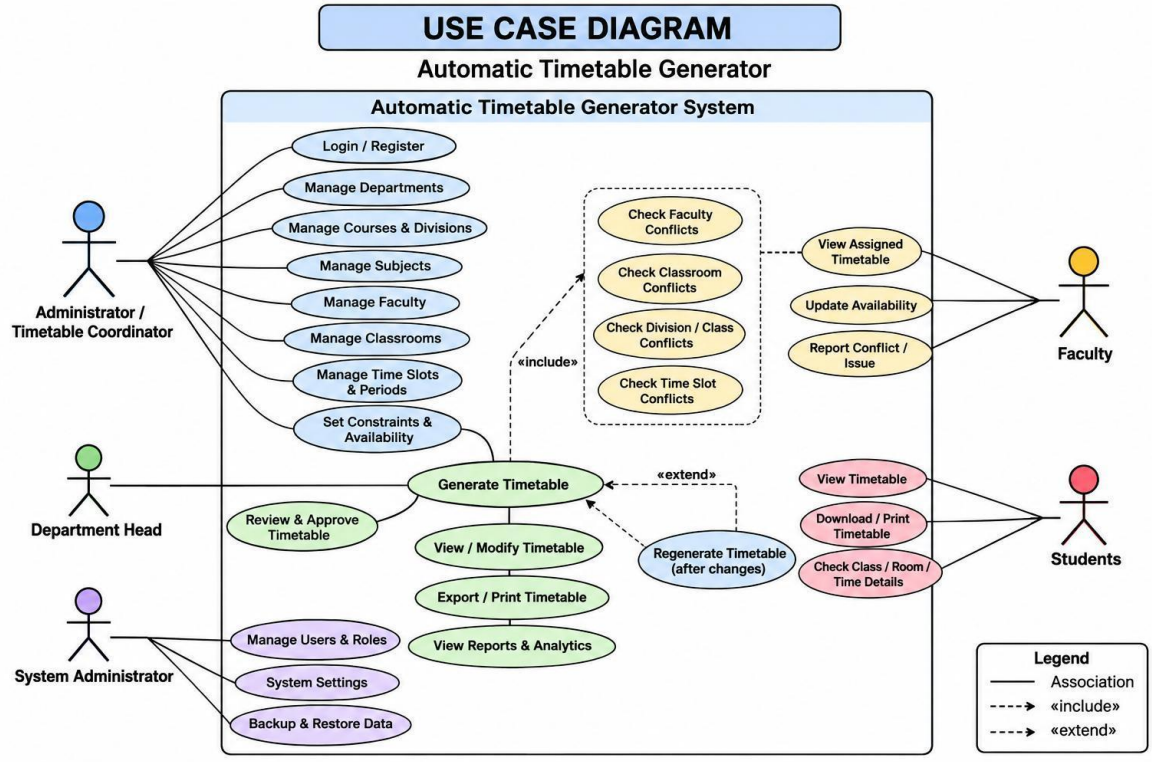


Fig 1: Use Case Diagram

Description: The administrator manages all master data and generates the timetable. Faculty members view their assigned schedules. Students view their division timetable. Department heads review and validate the timetable.

3.2. Class Diagram

Description: The main classes represent users, departments, faculty, divisions, subjects, classrooms, and timetable entries. A timetable entry links a subject, faculty member, division, classroom, day, and period.

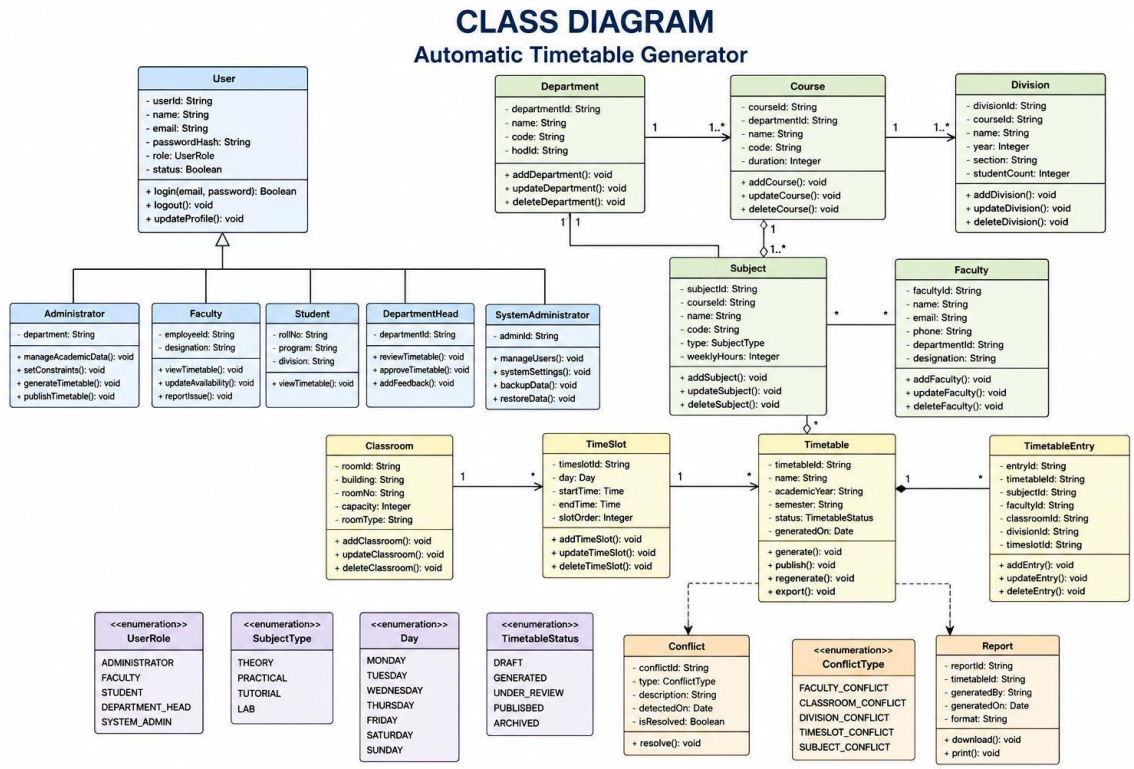


Fig 2: Class Diagram

3.3. Entity Relationship (ER) Diagram

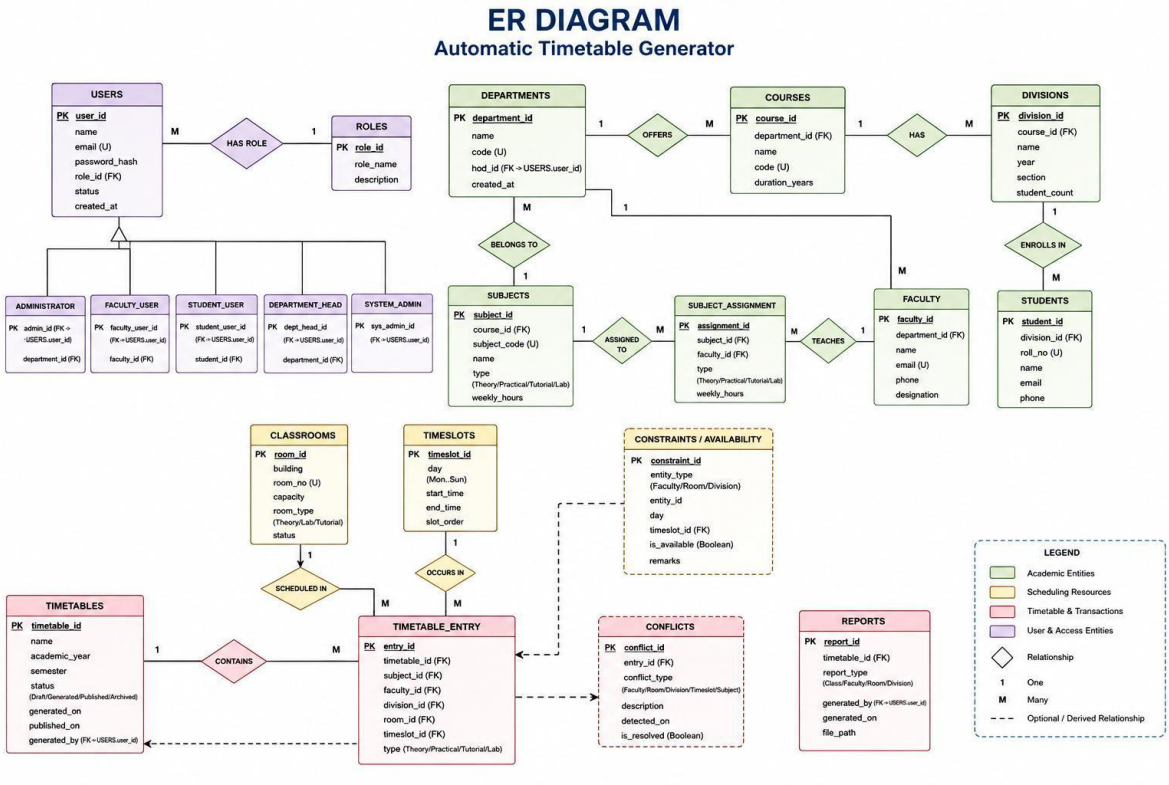


Fig 3: Entity Relationship (ER) Diagram

Description: The database stores relationships between departments, courses/divisions, subjects, faculty, classrooms, time slots, users, and timetable entries. The timetable entry acts as the central scheduling record.

CHAPTER 4

System Architecture Design

4.1. Frontend Architecture

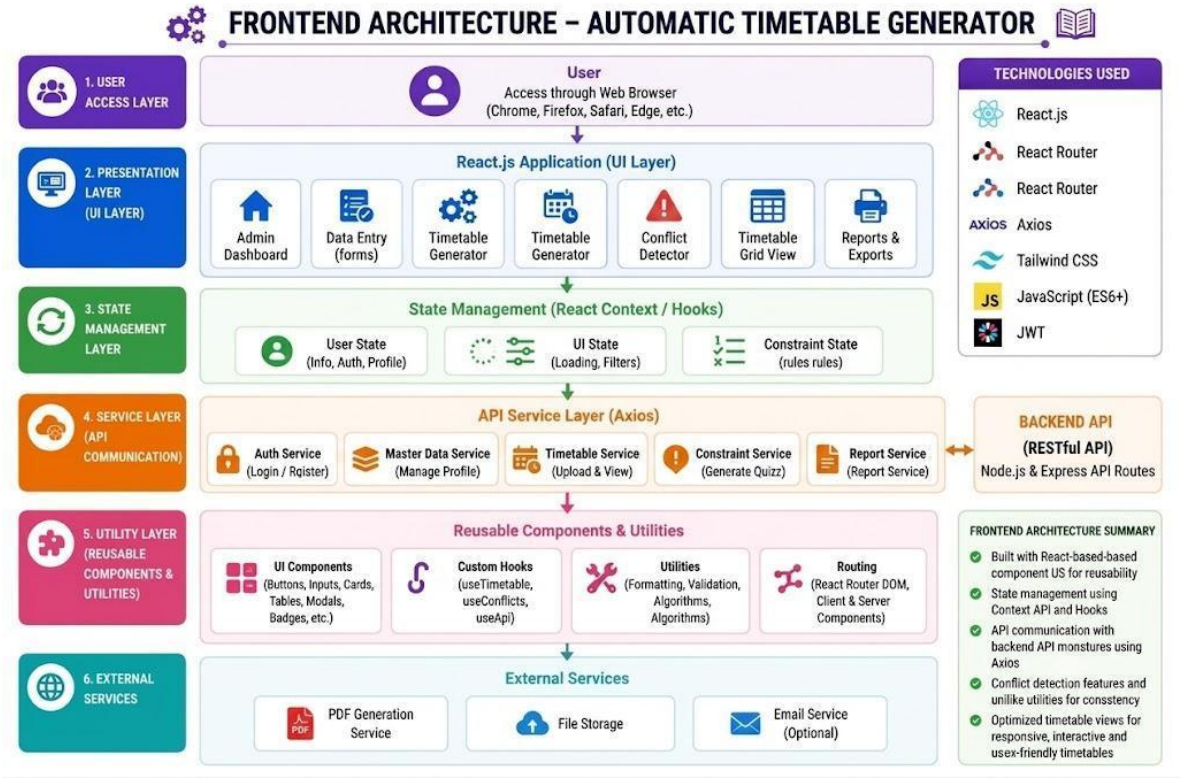


Fig 4: Frontend Architecture

4.2. Backend Architecture

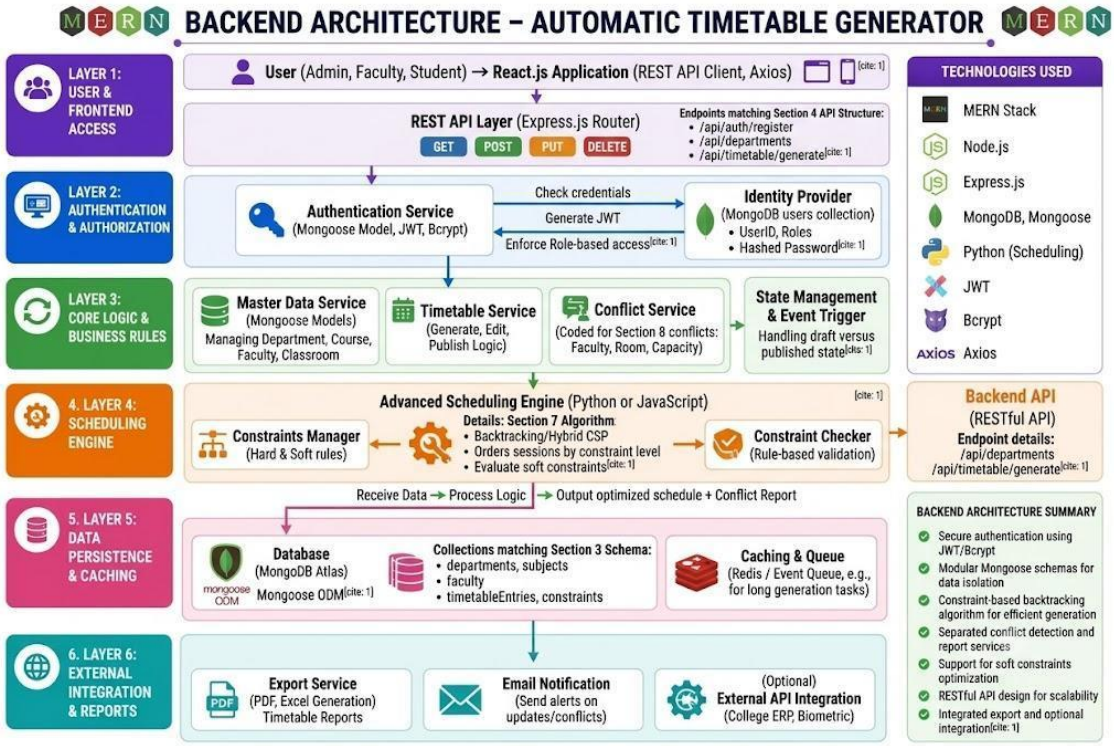


Fig 5: Backend Architecture

4.3. Database Schema Design

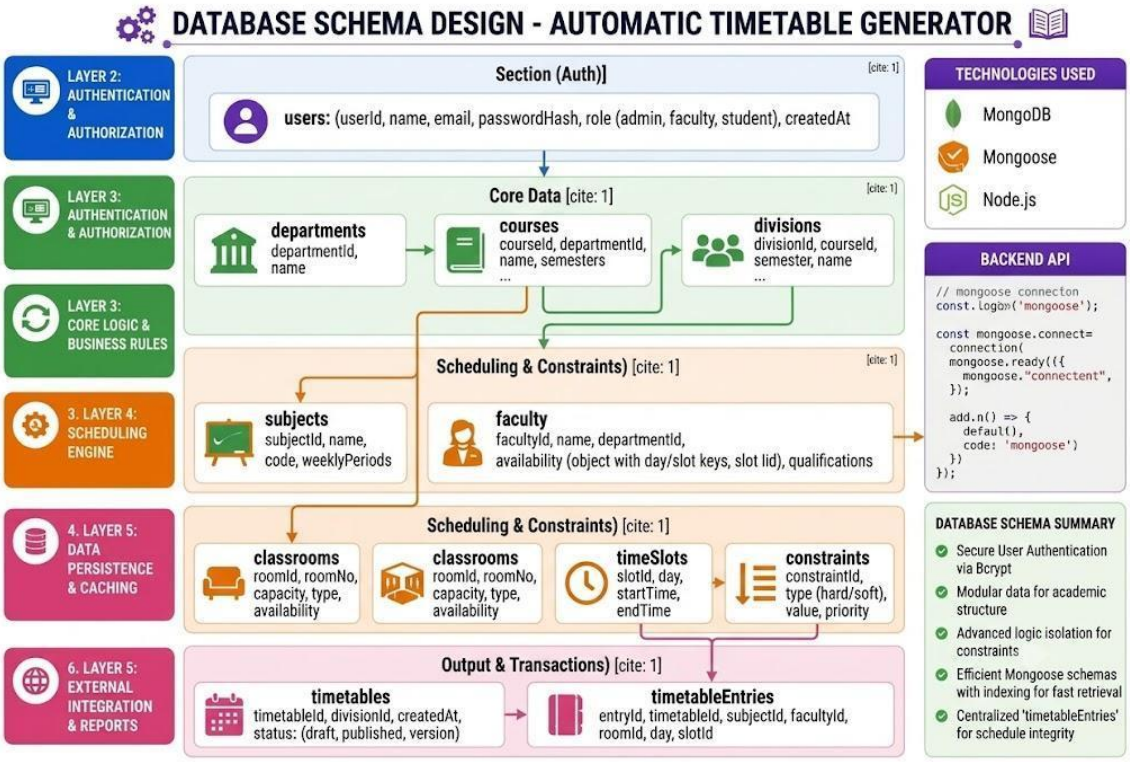


Fig 6: Database Schema Design

4.4. API Structure

A REST-based API can be used for communication between the frontend and backend. The following endpoints represent the proposed structure.

Table 10: Proposed API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register user
POST	/api/auth/login	Authenticate user
GET	/api/departments	List departments
POST	/api/departments	Create department
GET	/api/subjects	List subjects
POST	/api/subjects	Create subject
GET	/api/faculty	List faculty
POST	/api/faculty	Create faculty
GET	/api/classrooms	List classrooms
POST	/api/classrooms	Create classroom
GET	/api/timeslots	List time slots
POST	/api/constraints	Create scheduling constraint
POST	/api/timetable/generate	Generate timetable
GET	/api/timetable/:id	View generated timetable
PUT	/api/timetable/:id	Edit timetable
POST	/api/timetable/:id/publish	Publish timetable
GET	/api/timetable/:id/export	Export timetable

4.5. Security Considerations

- Passwords should be stored as secure hashes rather than plain text.
- JWT-based authentication should be used for protected APIs.
- Role-based authorization should prevent students or faculty from modifying administrative data.
- Input validation should be applied to all forms and API requests.
- Database access should be restricted to the backend service.
- Sensitive configuration values should be stored in environment variables.
- Audit information can be maintained for timetable generation and publication actions.
- Regular database backups should be performed in a deployed environment.

4.6. Timetable Generation Logic

The core of the project is the timetable generation engine. It receives subjects, faculty assignments, divisions, rooms, time slots, and constraints. The engine attempts to assign each required lecture to a valid combination of day, period, faculty, and classroom.

Hard Constraints

- A faculty member cannot teach two classes in the same period.
- A classroom cannot be allocated to two classes in the same period.
- A division cannot have two subjects in the same period.
- A faculty member cannot be assigned during an unavailable period.
- A classroom must satisfy the required capacity/type where applicable.
- Each subject should receive its required number of weekly periods.

Soft Constraints

- Avoid unnecessary consecutive lectures for the same faculty member.
- Distribute subject periods across the week where possible.
- Balance faculty workload across working days.
- Avoid excessive gaps in a division's daily schedule where possible.

4.7. Proposed Scheduling Algorithm

A constraint-based backtracking approach can be used for the academic prototype. The algorithm first creates a list of required class sessions and orders the most constrained sessions first. It then tries available time slots and resources, checks hard constraints, and backtracks whenever an invalid assignment is reached.

1. Load all divisions, subjects, faculty, classrooms, time slots, and constraints.

2. Expand each subject into the number of weekly class sessions required.

3. Sort sessions so that highly constrained sessions are scheduled first.

4. Select a session and find available day/period combinations.

5. Check faculty availability and existing faculty assignments.

6. Assign the first valid combination or evaluate alternative combinations.

7. If no valid combination exists, backtrack to the previous assignment.

8. Continue until all sessions are assigned or the system reports that the constraints cannot be satisfied.

9. Calculate a timetable quality score using soft constraints.

10. Save the best valid timetable and display it for review.

PSEUDOCODE

```
generateTimetable(data):
    sessions = expandSubjects(data.subjects)
    sessions = sortByConstraintLevel(sessions)
    schedule = emptySchedule()
    if backtrack(schedule, sessions, 0):
        score = evaluateSoftConstraints(schedule)
        return schedule, score
    else:
        return conflictReport

backtrack(schedule, sessions, index):
    if index == length(sessions):
        return true
    session = sessions[index]
    for option in availableOptions(session):
        if satisfiesHardConstraints(schedule, option):
            assign(schedule, session, option)
            if backtrack(schedule, sessions, index + 1):
                return true
            remove(schedule, session, option)
    return false
```

4.8. Conflict Detection

Conflict detection is an essential feature because the main objective of the system is to prevent timetable clashes. The system should validate each timetable entry against existing assignments.

Table 11: Conflict Types and System Actions

Conflict Type	Example	System Action
Faculty Conflict	Same teacher assigned to two classes at 10:00 AM	Reject assignment / report conflict
Room Conflict	Same room assigned to two divisions at the same time	Reject assignment / suggest another room
Division Conflict	Two subjects scheduled for the same division in one period	Reject assignment
Availability Conflict	Faculty is unavailable during selected period	Reject assignment
Capacity Conflict	Room capacity is smaller than class strength	Reject or suggest suitable room
Weekly Period Conflict	Subject receives fewer/more sessions than required	Flag timetable for correction

4.9. Expected System Output

- Conflict-free or conflict-minimized timetable.
- Division-wise timetable.
- Faculty-wise timetable.
- Classroom-wise timetable.
- List of unresolved constraints, if any.
- Timetable quality/validation summary.
- Printable/exportable timetable.

4.10. Testing Strategy

Table 12: Testing Strategy Matrix

Testing Type	Purpose
Unit Testing	Tests individual modules such as login, validation, and scheduling functions.
Integration Testing	Checks communication between frontend, backend, database, and scheduling engine.
System Testing	Validates the complete application workflow.
Algorithm Testing	Checks whether generated timetables satisfy hard constraints.
Performance Testing	Measures generation time for different numbers of classes and resources.
Security Testing	Checks authentication, authorization, input validation, and protected APIs.
User Acceptance Testing	Confirms that the timetable meets practical institutional requirements.

4.11. Sample Test Cases

Table 13: Sample Test Cases

Test Case	Input / Action	Expected Result
TC-01	Login with valid credentials	User is successfully logged in.
TC-02	Login with invalid password	Access denied with error message.
TC-03	Add a new subject	Subject is stored successfully.
TC-04	Assign same faculty to two classes at same period	System detects faculty conflict.
TC-05	Assign same room to two classes at same period	System detects room conflict.
TC-06	Generate timetable with valid data	System generates timetable.
TC-07	Faculty unavailable in selected period	Assignment is rejected.
TC-08	Room capacity below class strength	System rejects unsuitable room.
TC-09	Edit generated timetable	Authorized user can modify entry.
TC-10	Publish valid timetable	Timetable becomes available to users.

4.12. Advantages of the Proposed System

- Saves significant timetable preparation time.
- Reduces common scheduling errors.
- Improves utilization of faculty and classrooms.
- Provides centralized academic scheduling information.
- Makes timetable updates easier.
- Improves transparency for faculty and students.
- Provides a foundation for future optimization and analytics.

4.13. Limitations

- The quality of the timetable depends on accurate input data.
- Highly complex institutional rules may require additional algorithm development.
- Very large scheduling problems may require more advanced optimization techniques.
- The initial version may require manual review before final publication.
- The prototype may not include integration with existing college ERP systems.

4.14. Future Enhancements

- AI/ML-assisted timetable optimization.
- Automatic email and notification alerts.
- Mobile application for students and faculty.
- Integration with college ERP or student information systems.
- Examination timetable generation.
- Attendance and classroom utilization integration.
- Multi-campus timetable management.
- Advanced analytics and timetable quality scoring.
- Automatic detection of recurring scheduling preferences.
- Cloud-based multi-institution deployment.

4.15. Conclusion

The Automatic Timetable Generator is proposed as a practical solution to the challenges involved in manual academic scheduling. By organizing academic data and applying scheduling constraints, the system can automatically generate a structured timetable while reducing clashes and manual effort.

The project demonstrates the application of software engineering concepts including requirement engineering, feasibility analysis, Agile SDLC, UML modeling, database design, REST APIs, constraint-based scheduling, conflict detection, and software testing. The prototype can be further enhanced with advanced optimization, notifications, mobile access, and integration with institutional systems.

Chapter 5: Implementation

5.1 Frontend

The frontend of the Automatic Timetable Generator is a single-page web application built with plain HTML, CSS and JavaScript. It needs no installation or build tool; the application starts by opening index.html in any web browser. The whole interface is drawn by one render() function in app.js, which rebuilds the screen from a shared state object every time the user moves to another page or changes data. The main design and technical decisions are:

- **HTML, CSS and JavaScript (no framework):** index.html contains only an empty root container, a toast container and a dialog; every page is generated by JavaScript, which keeps the project small and easy to run.
- **Single state object:** the object S stores the login status, current page, search text, selected division, working days and the generated timetables. Calling render() redraws the screen from this state.
- **Mock data object:** the object D holds subjects, teachers, classrooms, divisions and time slots in memory. It is loaded with sample data of TY BSc Computer Science, Semester 5, Division A.
- **Reusable CRUD pages:** one crud() function together with the form-definition object F produces the Subjects, Teachers, Classrooms, Divisions and Time Slots pages, so the same code supports adding, editing, searching and deleting records.
- **Responsive and themed design:** CSS variables provide a light and a dark theme; on screens narrower than 760 px the sidebar turns into a menu opened with a button, and print styles hide the menu so only the timetable is printed.
- **Validation and feedback:** the login form is checked, all fields of the add/edit form are required, deletions need confirmation, and toast messages confirm each action.

Table 14: Frontend Pages and Their Main Functions

Page	Section	Main function(s)	Purpose
Login	5.1.1	login(), render()	Authenticates the administrator
Dashboard	5.1.2	page(), render()	Shows record counts and quick actions
Subjects	5.1.3	crud(), edit(), save(), del()	Manages subjects and weekly lecture counts
Teachers	5.1.4	crud(), edit(), save(), del()	Manages teachers and their subjects
Classrooms	5.1.5	crud(), edit(), save(), del()	Manages classrooms and labs
Divisions	5.1.6	crud(), edit(), save(), del()	Manages courses, semesters and divisions
Time Slots	5.1.7	crud(), edit(), save(), del()	Defines lecture periods and breaks
Generate Timetable	5.1.8	generate(), divSel()	Runs the scheduling algorithm
View Timetable	5.1.9	grid(), csv(), print()	Displays, prints and exports the timetable
Settings	5.1.10	theme()	Sets college name, academic year and theme

5.1.1 Login Page

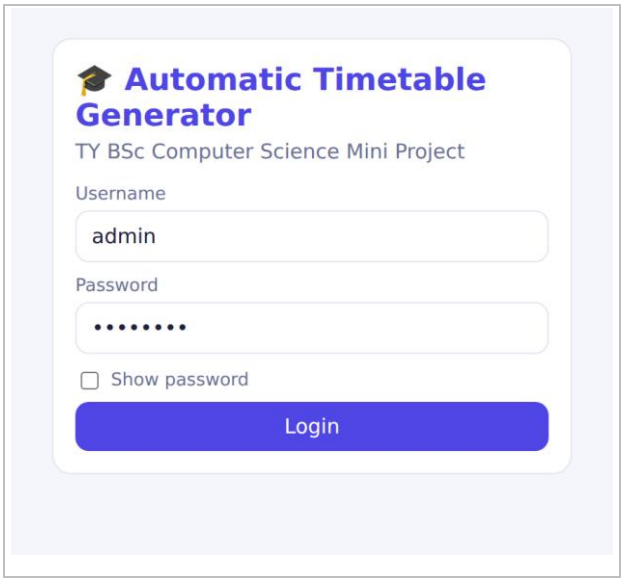


Fig 7: Login Page

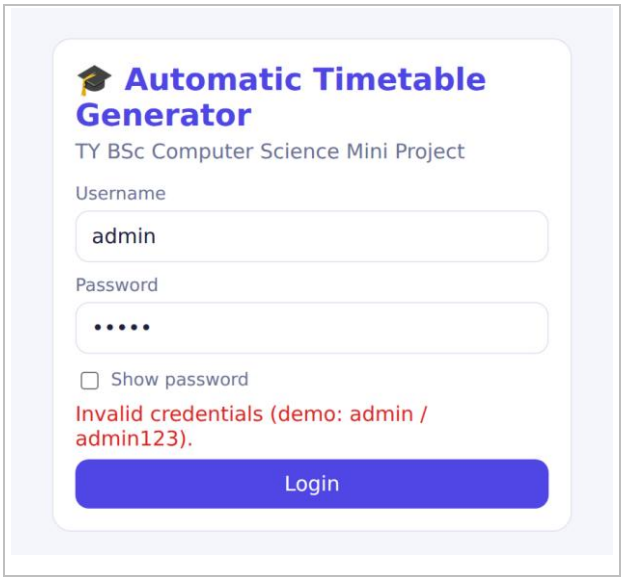


Fig 8: Login Page Showing an Invalid Credentials Error

5.1.2 Dashboard

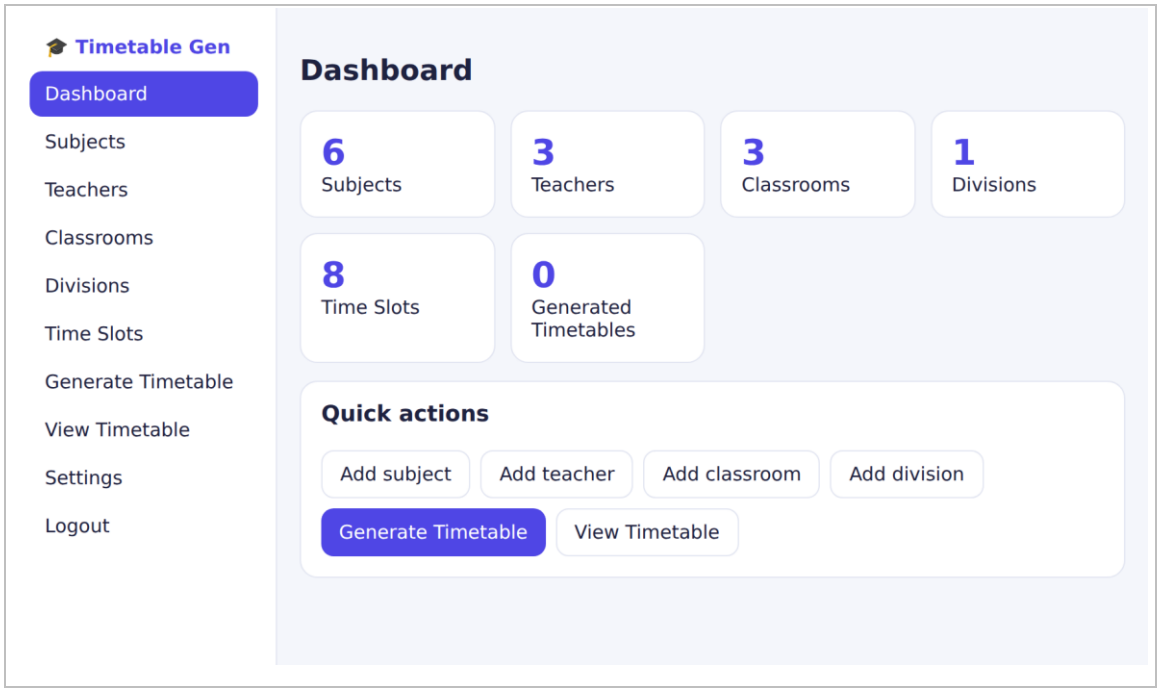


Fig 9: Dashboard

5.1.3 Subjects Page

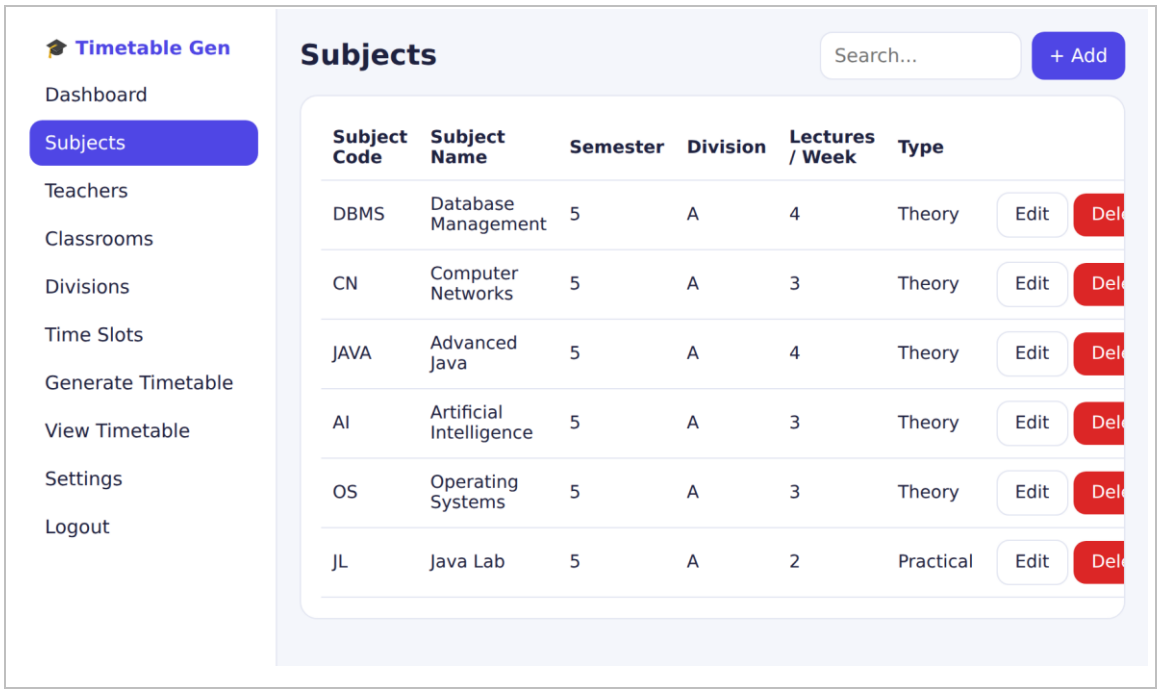


Fig 10: Subjects Page

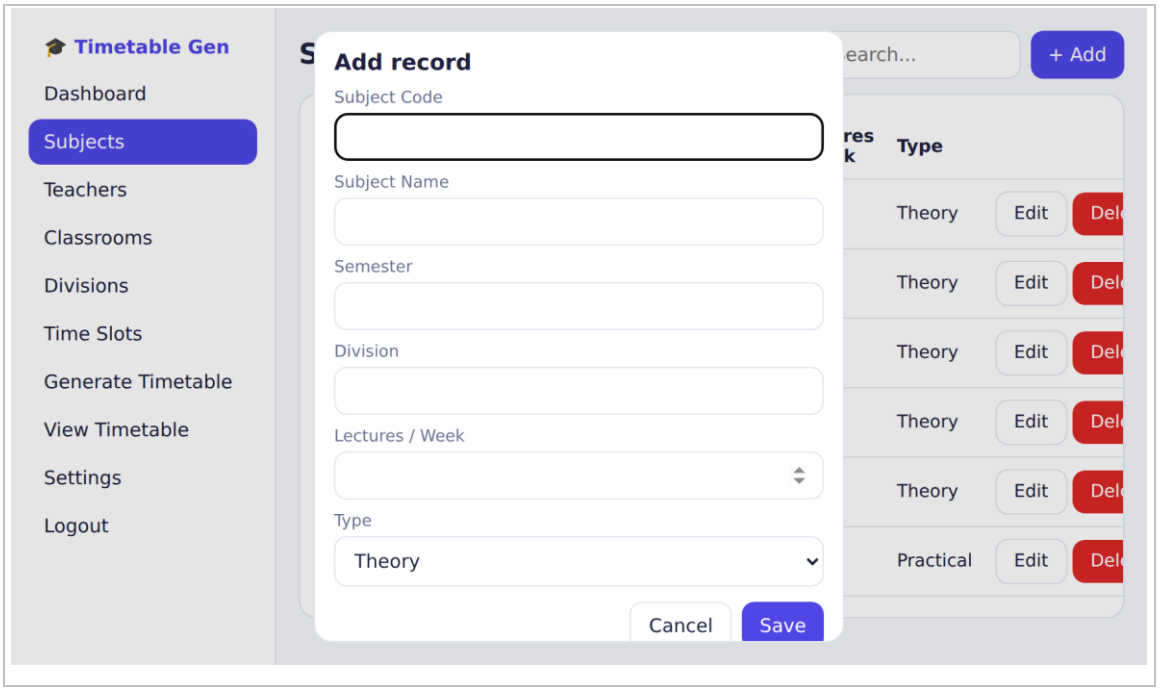


Fig 11: Add / Edit Record Dialog

5.1.4 Teachers Page

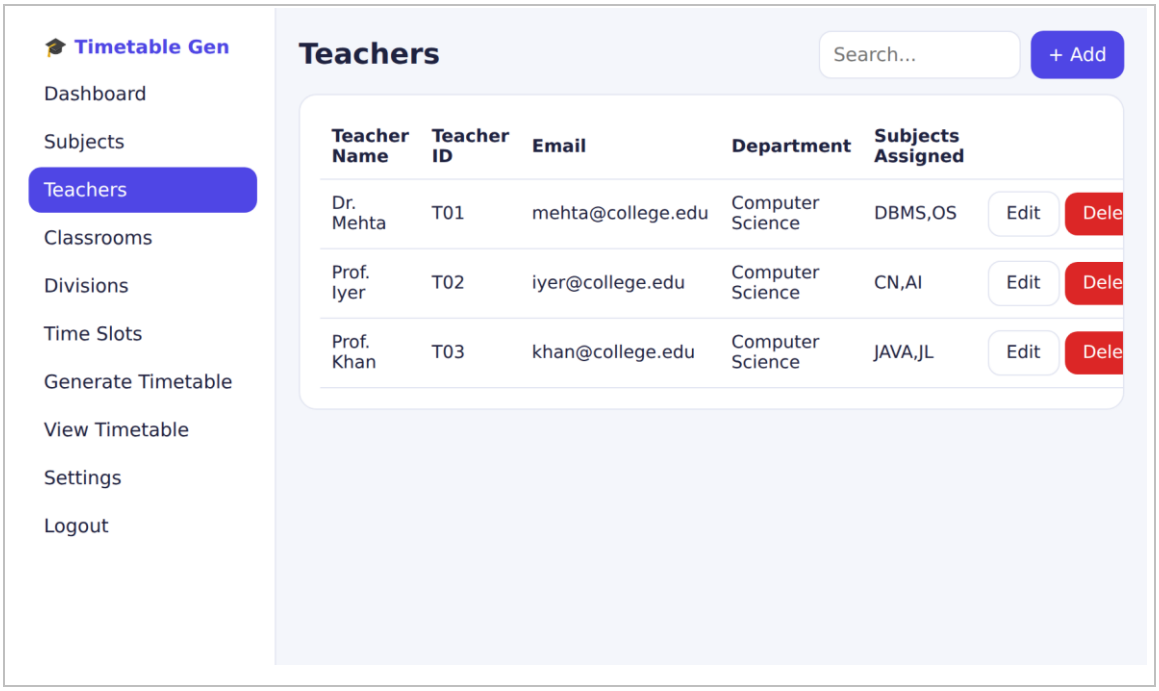


Fig 12: Teachers Page

5.1.5 Classrooms Page

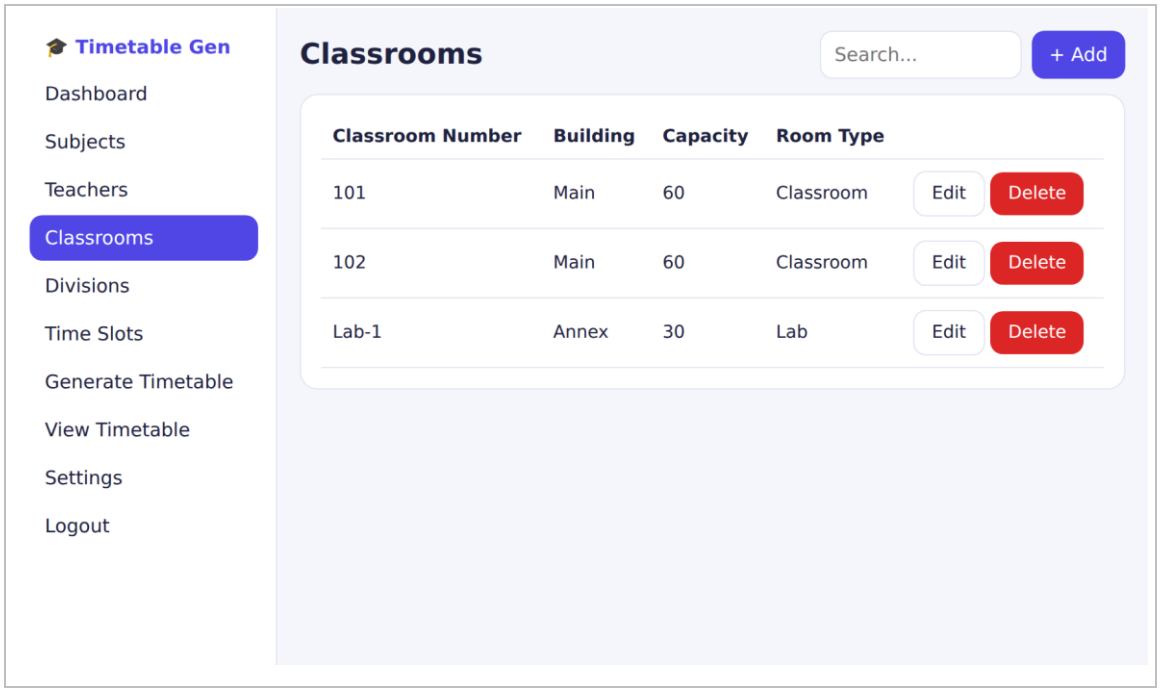


Fig 13: Classrooms Page

5.1.6 Divisions Page

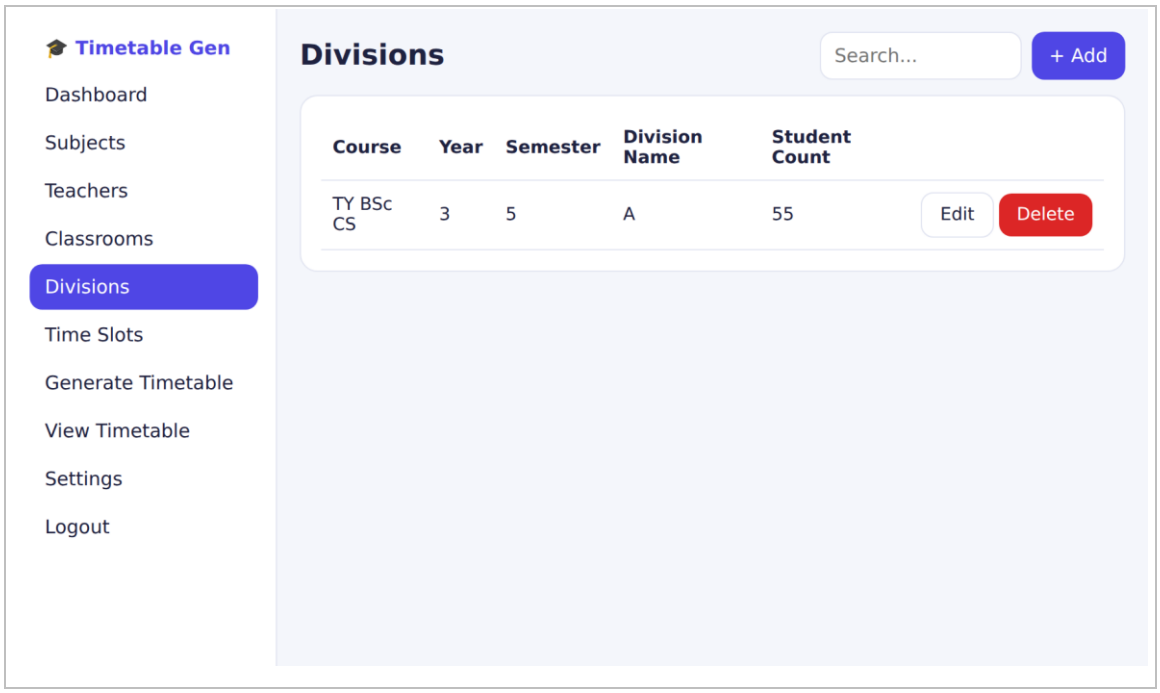


Fig 14: Divisions Page

5.1.7 Time Slots Page

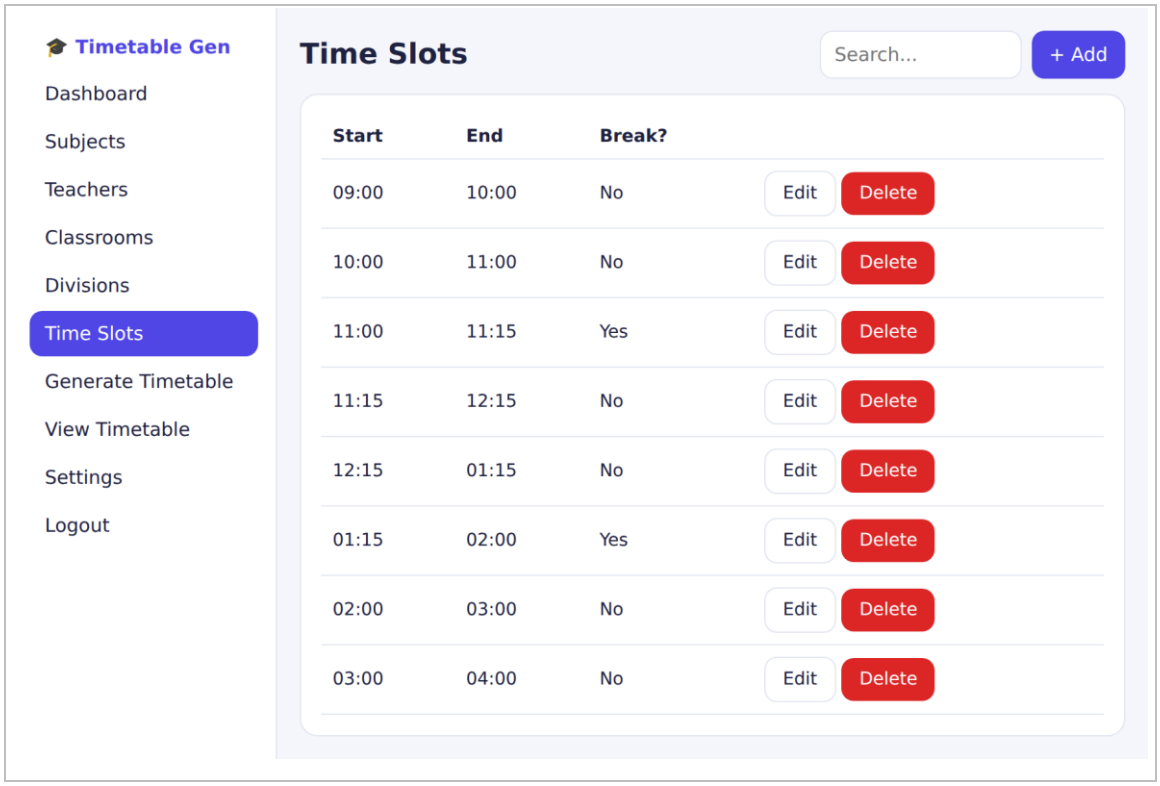


Fig 15: Time Slots Page

5.1.8 Generate Timetable Page

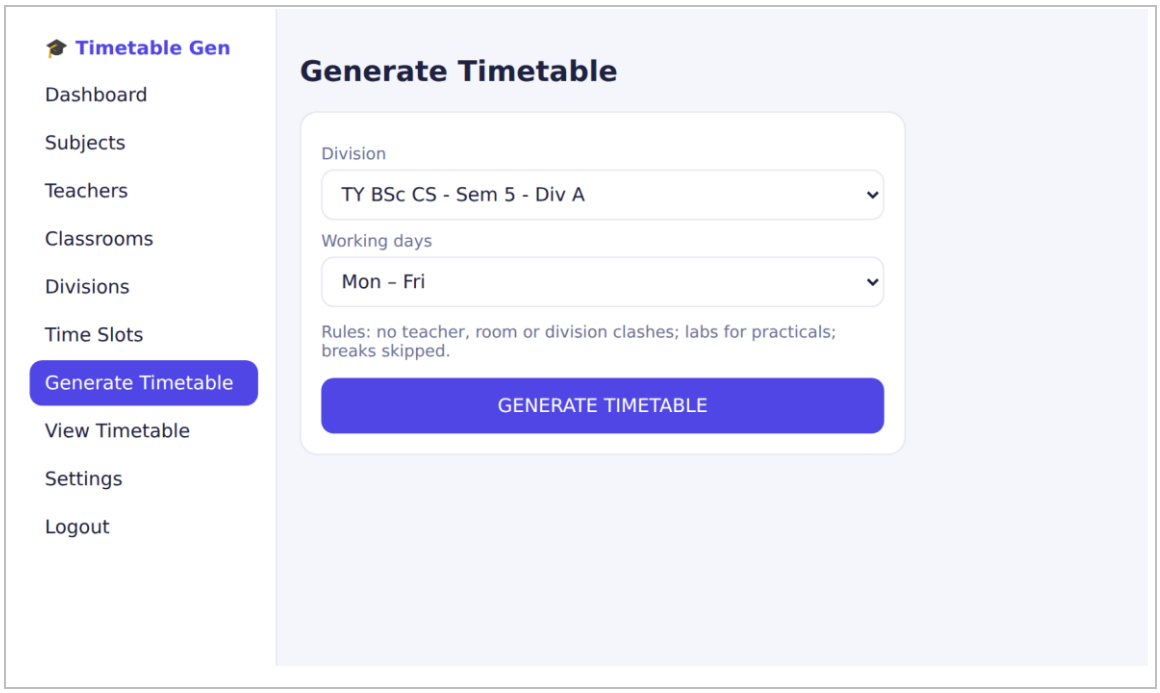


Fig 16: Generate Timetable Page

5.1.9 View Timetable Page

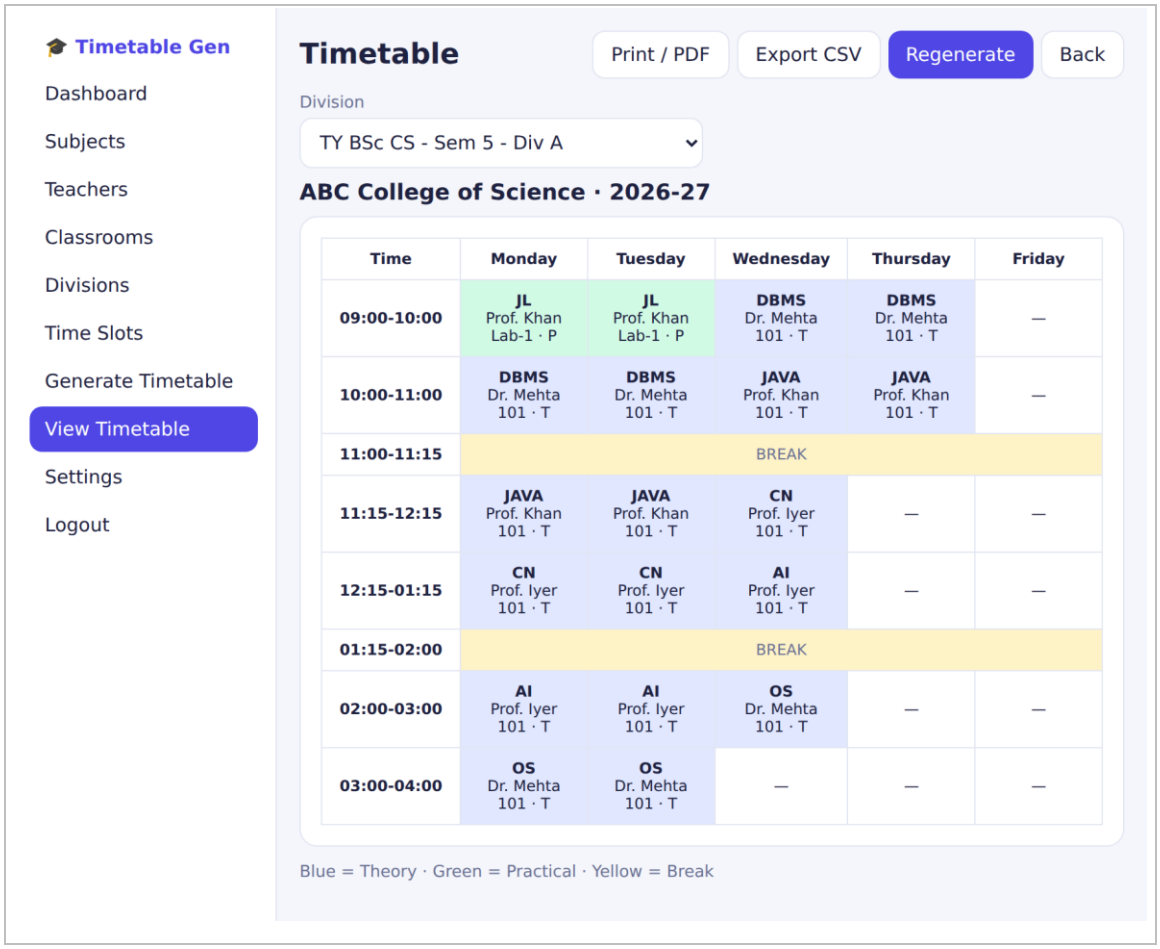


Fig 17: Generated Timetable (View Timetable Page)

5.1.10 Settings Page

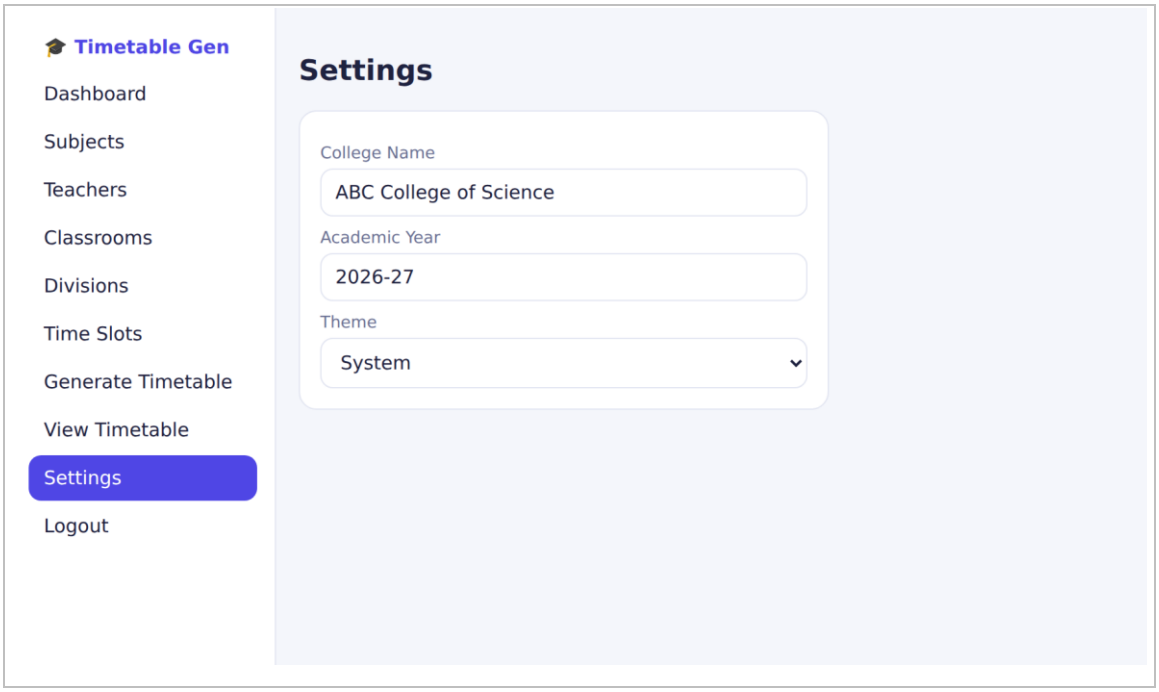


Fig 18: Settings Page.

5.1.11 Frontend Folder Structure

Fig 19: Frontend Folder Structure

Table 15: Frontend Files and Their Purpose

File	Purpose
index.html	Page structure: the root container, toast message area and the add/edit dialog, and links to the CSS and JavaScript files.
style.css	All styling: light and dark theme variables, sidebar layout, tables, timetable colours, dialog, responsive rules and print styles.
app.js	Sample data (D), form definitions (F), application state (S), login, CRUD functions, the timetable generation algorithm, timetable display, CSV export and page rendering.
README.txt	Instructions to run the project (open index.html) and the demo login details.

5.2 Backend

The backend of the Automatic Timetable Generator is a REST API built with Node.js, Express and MongoDB. It receives JSON requests, validates them, applies the application logic and returns JSON responses. The administrator is authenticated with a JSON Web Token (JWT), and the timetable is produced by a constraint-based scheduling algorithm that is kept in a separate service file so that it can be tested without a database. The frontend described in Section 5.1 works with in-memory sample data; the API described here provides the same operations (login, management of records and timetable generation) for a connected deployment. The main design and technical decisions are:

- **Node.js and Express 5:** a lightweight JavaScript server in which every feature is a route, a controller and, where needed, a middleware function.
- **MongoDB with Mongoose:** the data is stored as documents, and Mongoose schemas define the fields, the validation rules and the unique indexes. One Timetable document stores the complete week of one division.
- **JWT and bcrypt:** passwords are stored only as bcrypt hashes, and every protected route requires a valid token in the Authorization header.
- **express-validator and central error handling:** every input is checked before it reaches a controller, and every error is returned in the same JSON format.
- **Separated algorithm:** services/timetableGenerator.js is a pure function that does not access the database, so it can be tested on its own and explained easily.

Table 16: API Endpoints of the Backend

Area	Endpoints (base URL /api)	Access
Auth	POST /auth/register, POST /auth/login, GET /auth/me	Register: first admin only; me: token
Subjects	GET, POST /subjects; GET, PUT, DELETE /subjects/:id	JWT required
Teachers	GET, POST /teachers; GET, PUT, DELETE /teachers/:id	JWT required

Area	Endpoints (base URL /api)	Access
Classrooms	GET, POST /classrooms; GET, PUT, DELETE /classrooms/:id	JWT required
Divisions	GET, POST /divisions; GET, PUT, DELETE /divisions/:id	JWT required
Time slots	GET, POST /timeslots; PUT, DELETE /timeslots/:id	JWT required
Timetable	POST /timetable/generate, POST /timetable/regenerate, GET /timetable, GET /timetable/:id, GET /timetable/division/:division, DELETE /timetable/:id	JWT required
Dashboard	GET /dashboard/stats	JWT required

5.2.1 Server Setup and Database Connection

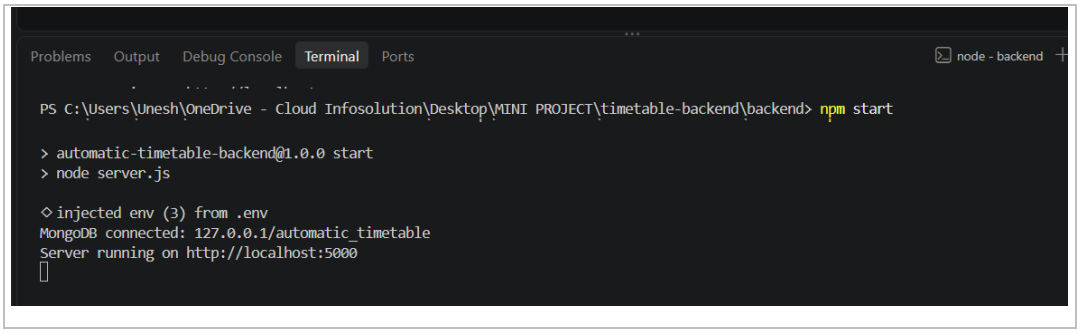


Fig 20: Backend Server Running and Connected to MongoDB

Opening http://localhost:5000 in the browser returns a short JSON message, which confirms that the API is running.

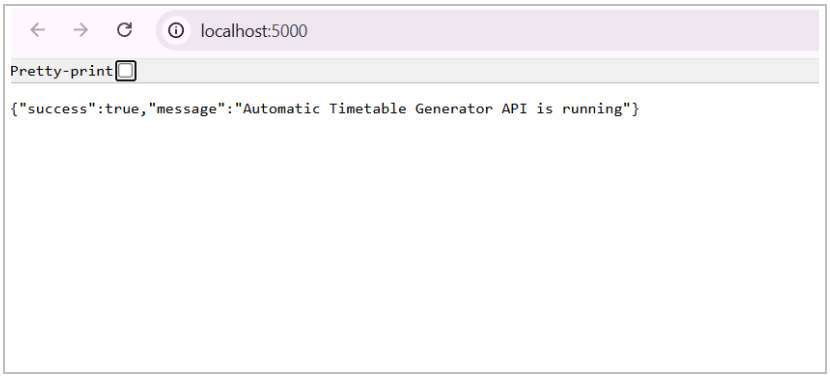


Fig 21: API Root Route Responding in the Browser

5.2.2 Database Models

The data is organised into seven Mongoose models: User, Teacher, Subject, Classroom, Division, TimeSlot and Timetable. Each schema lists the required fields, the allowed values (for example the subject type can only be Theory or Practical) and the unique indexes that prevent duplicate records, such as the same subject code twice in one division.

Table 17: Collections of the MongoDB Database

Collection	Main fields	Rule
User	name, email, password, role	Email unique; password hashed and hidden from queries
Teacher	teacherName, teacherId, email, department, subjects[]	teacherId and email unique
Subject	subjectName, subjectCode, semester, course, division, type, lecturesPerWeek, teacher, duration	Unique per code, course, semester and division
Classroom	roomNumber, building, capacity, type	roomNumber unique; type is Classroom or Laboratory
Division	course, year, semester, divisionName, studentCount	Unique per course, year, semester and name
TimeSlot	day, startTime, endTime, isBreak	Unique per day and start time
Timetable	course, year, semester, division, entries[]	One document per division

Key points

- **References:** subject, teacher and classroom inside a timetable entry store the id of another document; Mongoose populate() replaces the ids by names when the timetable is read.
- **Embedded entries:** all lectures of a division are stored inside one Timetable document, so a whole week is loaded, replaced or deleted with a single operation.

- **Time format:** times are stored as 24-hour text such as 13:00, which can be sorted and compared directly.

5.2.3 Authentication and Authorization

Only the administrator can use the API. The first administrator can register freely; after that, a new administrator can be created only by a logged-in administrator. The User model hashes the password with bcrypt before saving it, and the password field is excluded from normal queries (select: false). On a successful login the server signs a JWT that contains the user id and sends it back; the client sends it in the Authorization header of every later request, where the protect middleware verifies it.

5.2.4 Input Validation and Error Handling

Every request body is validated with express-validator before the controller runs. Each route has a list of rules (for example subjectRules), followed by one validate function that stops the request with status 400 and a readable message if any rule fails. Errors that happen later, such as a duplicate record or an invalid id, are converted into the same JSON format by one central error handler.

5.2.5 Timetable Generation Algorithm

The scheduling logic is in services/timetableGenerator.js. The problem is treated as a constraint satisfaction problem: every lecture must be placed in a day and time slot so that no rule is broken. The algorithm is a randomised greedy search with restarts. It places the subjects that need the most slots first, tests every free position against all constraints, and prefers the day on which the subject appears least so that lectures are spread over the week. If one subject cannot be placed, the whole attempt is discarded, the order is shuffled and the search starts again, up to 200 attempts. This simple form of backtracking is fast and, when the data is not sufficient, it ends with a clear error message.

Table 18: Scheduling Rules and How They Are Enforced

Rule	Enforcement in the code
1 Teacher conflict	A set of teacherId|day|time is kept; a slot is rejected if the teacher is already in the set. It is pre-filled with the lectures of other divisions.
2 Classroom conflict	A set of roomId|day|time is kept in the same way and is also pre-filled from other divisions.
3 Division conflict	A set of day|time for the current division; a division cannot have two lectures at once.
4 Lecture count	A subject is placed until it has exactly lecturesPerWeek sessions; verifySchedule() checks the count at the end.
5 Practical in laboratory	Practical subjects can see only rooms of type Laboratory.
6 Theory in classroom	Classrooms are tried first; laboratories are only a fallback. Rooms that are large enough for the division are preferred.
7 Break time	Slots with isBreak set to true are never used.
8 Working days	Only days that have configured time slots are considered.

Key points

- **Pure function:** buildSchedule() receives plain arrays and sets and returns a result object; it never reads or writes the database.
- **Hardest first:** subjects that need more slots are placed first, because they are the most difficult to fit later.
- **Spreading:** among all valid positions, the one on the day with the fewest lectures of that subject is chosen.
- **Multi-slot sessions:** a subject with duration 2, such as a practical, needs two consecutive slots that touch each other in time.
- **Limitation:** the search is greedy, so it is not guaranteed to find a timetable whenever one exists; when it fails, the error message names the problem subject.

5.2.6 Sample Data and Automated Testing

Output of npm test

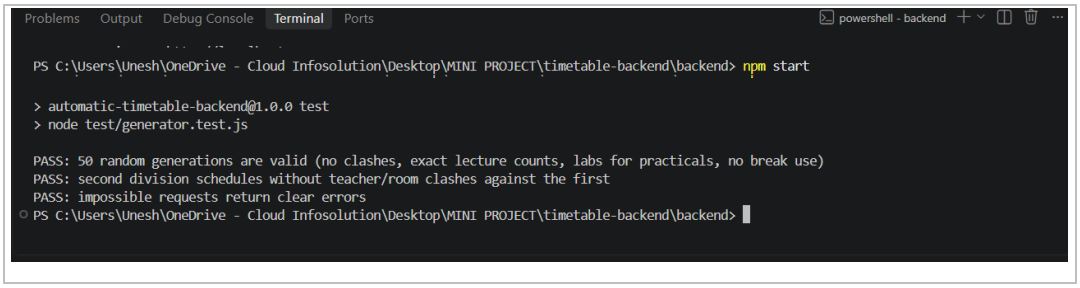


Fig 22: Output of npm test

Chapter 6: Integration & System Testing

6.1 Introduction

Testing was carried out to confirm that the parts of the Automatic Timetable Generator work correctly together and that the finished application meets its requirements. Two levels of testing are covered in this chapter:

- **Integration testing:** checks the places where two parts meet: the routes, validation, authentication, controllers, models and the MongoDB database of the backend; the scheduling algorithm and the timetables of other divisions; and the Generate and View pages of the frontend.
- **System testing:** checks the complete web application from the administrator's point of view, from the login page to the generated timetable and its export.

The screens and the API were tested manually (black-box testing): the pages in the web browser and the API through Postman. The scheduling algorithm is randomised, so it was also tested with automated scripts (npm test and test/extra.test.js) that repeat the generation many times and check every result with the independent verifySchedule() function. As described in Chapter 5, the frontend works with sample data kept in the browser, while the backend stores its data in MongoDB; each was therefore tested through its own interface.

6.2 Test Environment

Table 19: Test Environment

Component	Details
Frontend	HTML, CSS and JavaScript opened in a web browser (index.html); no installation or build tool is needed.
Backend	Node.js and Express, running at http://localhost:5000 (started with npm start or npm run dev).
Database	MongoDB running locally at 127.0.0.1:27017; database automatic_timetable accessed through Mongoose.
Test data	Created with npm run seed: one administrator, 5 teachers, 3 classrooms, 2 laboratories, 8 subjects and the time slots of Monday to Saturday (09:00 to 16:00, lunch break 13:00 to 14:00).

Component	Details
Tools	Web browser, Postman, MongoDB Compass, VS Code and the terminal.
Test method	Manual black-box testing of the screens and the API, plus automated scripts for the scheduling algorithm.

6.3 Integration Testing

6.3.1 Integration Points

The table below lists the places where two parts of the system meet. Each of them was tested.

Table 20: Integration Points

Integration point	How the parts interact
Routes and validation	Each route runs its express-validator rules first; invalid data is stopped with status 400 before the controller is reached.
Authentication and protected routes	The protect middleware reads the Bearer token, verifies it with the JWT secret and loads the user; without a valid token the request ends with 401.
Controllers and MongoDB	Controllers read and write the seven collections through the Mongoose models; duplicate and invalid values are turned into 409 and 400 responses by the error handler.
Timetable controller and algorithm	The controller loads subjects, time slots, classrooms and teachers, passes them to buildSchedule(), checks the result with verifySchedule() and saves it.
Algorithm and other divisions	Teachers and rooms already used by saved timetables are loaded as busy sets, so a new division cannot double-book them.
Frontend Generate and View pages	generate() fills the timetable object in the application state and opens the View Timetable page, which draws the grid and offers print and CSV export.

6.3.2 Integration Test Cases

IT stands for Integration Test. The figures named in the Actual Result column are the screenshots that show the result. Tests IT-07 to IT-11 were run by the automated scripts, and their output is given in Section 6.3.3.

Table 21: Integration Test Cases

ID	Modules	Test scenario	Expected result	Actual result	Status
IT-01	Auth API, User model, MongoDB	POST /api/auth/login with admin@college.com and the correct password	200 OK; JWT token and user details (id, name, email, role); no password	Token and user returned without a password (Fig 6.1)	**Pass**
IT-02	Auth API, bcrypt	Log in with a wrong password	401; message "Invalid email or password"	Message returned (Fig 6.2)	**Pass**
IT-03	Auth middleware, Subject routes	GET /api/subjects without a token	401; message "Not authorized. Please log in."	401 returned (Fig 6.3)	**Pass**
IT-04	Validators, Subject routes	POST /api/subjects with an empty body (token sent)	400; message "Subject name is required" and a list of the errors	400 with the validation message	[Pass/Fail]
IT-05	Timetable API, algorithm, MongoDB	POST /api/timetable/generate for BSc CS, TY, Semester 5, Division A	201; entries with subject, teacher and classroom; timetable saved in MongoDB	Timetable returned and stored	[Pass/Fail]
IT-06	Timetable API, MongoDB	Send the same generate request again	409; "A timetable already exists for this division. Use /api/timetable/regenerate to replace it."	409 returned	[Pass/Fail]

ID	Modules	Test scenario	Expected result	Actual result	Status
IT-07	Algorithm, saved timetables of other divisions	Schedule a second division after the first, and then three divisions in a row	No teacher or room clash; every result passes verifySchedule()	No clash in any division; each needed only one attempt (Section 6.3.3)	**Pass**
IT-08	Algorithm, room allotment	Schedule practical subjects that need two slots	Each practical is in a laboratory, in two back-to-back slots of the same lab	Correct in 50 of 50 random runs (Section 6.3.3)	**Pass**
IT-09	Algorithm, pre-checks	Require 40 slots when only 36 teaching slots exist	Clear error message about insufficient slots	“Insufficient time slots: subjects need 40 slots per week but only 36 teaching slots exist.”	**Pass**
IT-10	Algorithm, pre-checks	Schedule a subject that has no teacher	Clear error message naming the subject	“No teacher is assigned to subject "DBMS".”	**Pass**
IT-11	Algorithm, working days	Give only Monday to Friday slots	No lecture is placed on Saturday	All lectures placed on Monday to Friday (Section 6.3.3)	**Pass**
IT-12	Frontend Generate page, application state, View page	Select a division and press GENERATE TIMETABLE	Spinner, then the View Timetable page with the weekly grid	Grid displayed with the generated lectures (Fig 5.10, Fig 5.11)	**Pass**

6.3.3 Test Evidence

The screenshots below show the results of the API tests (IT-01 to IT-03).



Fig 23: Successful Login with the JWT Token (IT-01)

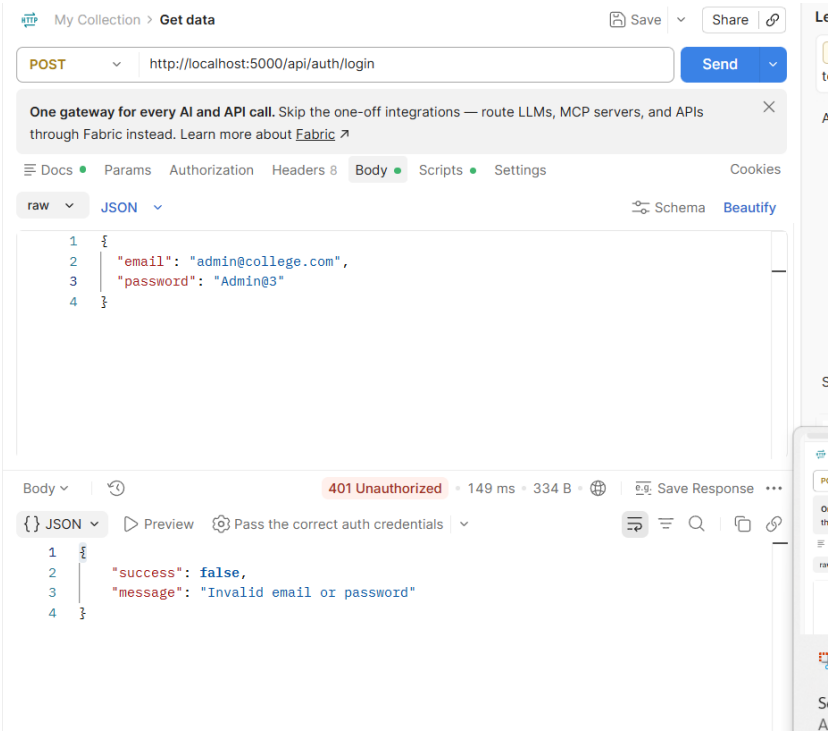


Fig 24: Login with a Wrong Password (IT-02)

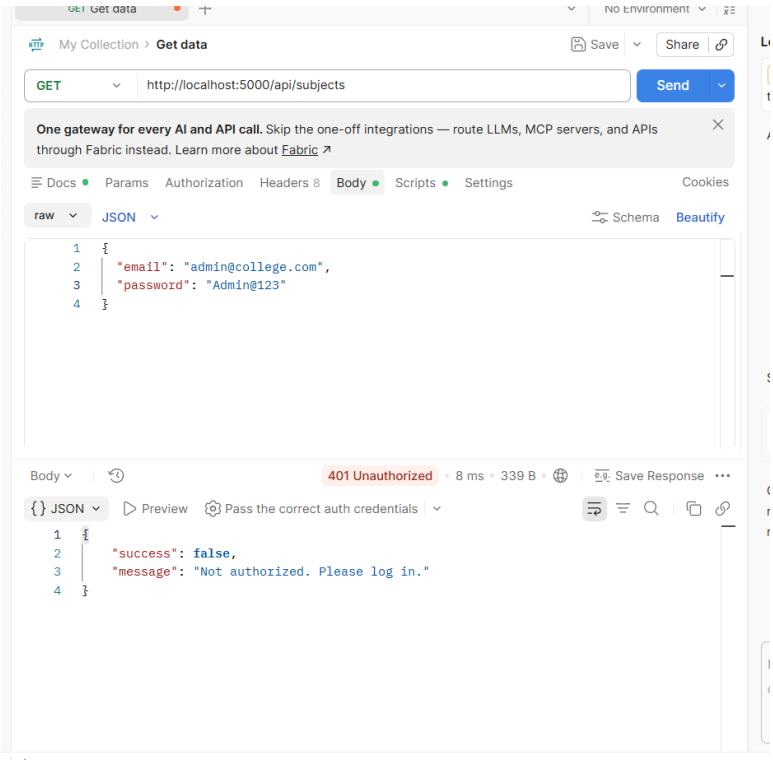


Fig 25: 401 Response When No Token Is Sent (IT-03)

The tests of the scheduling algorithm (IT-07 to IT-11) were run from the terminal. The script test/generator.test.js (npm test) was shown in Fig 5.16. The additional script test/extra.test.js gave the following output:

Output of node test/extra.test.js

PASS: practicals use one laboratory for two back-to-back slots (50 random runs)
PASS: three divisions scheduled one after another without any clash
PASS: every subject receives exactly lectures x duration slots (26 slots in total)
PASS: with Monday-Friday slots only, no lecture is placed on Saturday
PASS: a subject without a teacher is rejected with a clear message

6.4 System Testing

6.4.1 System Test Cases

ST stands for System Test. These cases check the finished application as a whole against its requirements. The figures named in the Actual Result column are the screenshots from Chapter 5 and Chapter 6 that show the result.

Table 22: System Test Cases

ID	Area	Test scenario	Expected result	Actual result	Status
ST-01	Login page	Open index.html in the browser	Login card with username, password, Show password and Login button	Login page displayed (Fig 5.1)	**Pass**
ST-02	Login validation	Press Login with wrong credentials	Error message is shown and the user stays on the login page	"Invalid credentials (demo: admin / admin123)." shown (Fig 5.2)	**Pass**
ST-03	Dashboard	Log in with admin and admin123	Summary tiles with the record counts and the Quick actions panel	6 subjects, 3 teachers, 3 classrooms, 1 division, 8 time slots and 0 timetables shown (Fig 5.3)	**Pass**
ST-04	Subjects page	Open Subjects from the menu	Table of subjects with Edit and Delete buttons	Six subjects listed (Fig 5.4)	**Pass**
ST-05	Add record	Press + Add on the Subjects page	Dialog with all the fields of a subject	Add record dialog displayed (Fig 5.5)	**Pass**
ST-06	Generate page	Open Generate Timetable	Division and working-days selectors, the rules note and the GENERATE button	All items displayed (Fig 5.10)	**Pass**

ID	Area	Test scenario	Expected result	Actual result	Status
ST-07	Timetable generation	Press GENERATE TIMETABLE	Weekly grid with theory in blue, practicals in green and breaks in yellow	Grid displayed with the three colours (Fig 5.11)	**Pass**
ST-08	Breaks	Inspect the generated grid	No lecture is placed in a BREAK row	BREAK rows contain no lecture (Fig 5.11)	**Pass**
ST-09	Export	Press Export CSV on the View Timetable page	A CSV file with the Time column and one column per day is downloaded	CSV file downloaded with the Time column and one column per day	[Pass/Fail]

6.4.2 End-to-End Scenario

This scenario follows the administrator through the whole application. It checks that the modules work together in the order a real user would use them.

- **Step 1:** Open index.html and log in with admin and admin123.
- **Step 2:** Check the record counts on the Dashboard.
- **Step 3:** Open the Subjects page and check the list of subjects.
- **Step 4:** Open Generate Timetable, select the division and the working days, and press GENERATE TIMETABLE.
- **Step 5:** Check the weekly grid on the View Timetable page.
- **Step 6:** Press Export CSV and open the downloaded file.

Expected result: every step succeeds and the timetable is displayed without any lecture in a break slot. Actual result: steps 1 to 5 are shown in Fig 5.1, Fig 5.3, Fig 5.4, Fig 5.10 and Fig 5.11, and step 6 downloads the CSV file (ST-09).

6.4.3 System Test Evidence

The screenshots of the login, dashboard, generate and view pages are given in Chapter 5 (Fig 5.1 to Fig 5.11).

6.5 Observations

- **Conflict-free results:** the generated timetables had no teacher, room or division clash; 50 random generations and three divisions scheduled one after another were all valid.
- **Quick generation:** on the sample data one attempt was enough for each of the three divisions, and no more than two attempts were needed in 50 random runs, so the result appears almost instantly.
- **Clear error messages:** when the data is not sufficient (too few time slots, no laboratory, no teacher), the algorithm stops with a message that names the problem instead of producing a wrong timetable.
- **Access control:** protected API endpoints reject a missing or invalid token with 401, and invalid input is rejected with 400 before it reaches the database.
- **Limitations:** the tests were manual for the screens and the API; the frontend currently uses its own sample data and is not yet connected to the API; and the greedy algorithm is not guaranteed to find a timetable whenever one exists. Automated API tests, a connected frontend and testing on more browsers and mobile screens can be added in future.

In summary, the tests show that the backend modules, the scheduling algorithm and the web pages work together as designed, and that the generated timetables satisfy the scheduling rules.

Conclusion

The Automatic Timetable Generator was developed to remove the slow and error-prone work of preparing a college timetable by hand. A timetable made manually often has a teacher in two places at once, a laboratory used twice, or a subject with too few lectures. The project studied this problem, planned the work, designed the system and then built and tested a working application that creates a conflict-free weekly timetable automatically.

The final system consists of a web frontend and a backend. The frontend is built with plain HTML, CSS and JavaScript and gives the administrator a login page, a dashboard, pages to manage subjects, teachers, classrooms, divisions and time slots, a page to generate the timetable, and a weekly timetable view that can be printed or exported as a CSV file. The backend is a REST API built with Node.js, Express and MongoDB. It protects its endpoints with JWT authentication, stores passwords as bcrypt hashes, validates every input, and keeps the data of subjects, teachers, classrooms, divisions, time slots and timetables in MongoDB.

The core of the project is the scheduling algorithm. It treats the timetable as a constraint satisfaction problem and places every lecture so that no teacher, classroom or division is used twice at the same time, practical subjects are placed in laboratories for two back-to-back slots, break slots stay empty, and each subject receives exactly its weekly number of lectures. Teachers and rooms already used by other divisions are taken into account, and every result is checked again by an independent verification function before it is saved.

Testing showed that the algorithm works as designed. In the automated tests, 50 random generations were all valid, three divisions were scheduled one after another without any clash, and no more than two attempts were needed to find a valid timetable on the sample data. When the data is not sufficient, for example when there are too few time slots, no laboratory or no teacher, the system stops with a clear message instead of producing a wrong timetable. The screens of the application were tested in the browser as described in Chapter 6.

Overall, the project met its main objective of generating a clash-free timetable automatically and quickly. It also showed that a simple, low-cost technology stack is enough to build a useful tool for a college, and it gave the author practical experience in full-stack development, REST API design, authentication, database design, algorithm design and software testing.

Future Scope

The present system covers the core work of timetable generation. The following enhancements can be added in future versions to make it more useful, more flexible and ready for wider use:

- **Connecting the frontend to the API:** replace the sample data kept in the browser with calls to the backend, so that the administrator's records and the generated timetables are stored in MongoDB and shared by all users.
- **Teacher availability and workload:** allow teachers to mark days or hours when they are not available, and set a maximum number of lectures per teacher per day.
- **Lab batches:** split a division into batches so that practicals of different batches run in different laboratories at the same time.
- **Manual editing:** let the administrator move a lecture by drag and drop, with the system checking for clashes immediately.
- **Better optimisation:** use backtracking or a genetic algorithm to find a timetable in difficult cases where the present greedy method fails, and add preferences such as no free gaps between lectures.
- **More export options:** export the timetable as a formatted PDF or Excel file, in addition to the print and CSV options.
- **Roles for teachers and students:** give teachers and students their own login to view their personal timetable, and send a notification when the timetable changes.
- **Several courses and years:** manage many courses, years and divisions of a whole college in one place, with exam and holiday schedules.
- **Automated testing:** add automated API and user-interface tests, and test on more browsers and mobile screens, to replace the present manual tests.
- **Cloud deployment:** host the application on a cloud platform with a managed database so that it can be used by the whole institution.

References

[1] OpenJS Foundation. “Node.js Documentation.” https://nodejs.org/docs/latest/api/

[2] OpenJS Foundation. “Express – Node.js Web Application Framework.” https://expressjs.com

[3] MongoDB, Inc. “MongoDB Documentation.” https://www.mongodb.com/docs/

[4] Mongoose. “Mongoose Documentation.” https://mongoosejs.com/docs/

[5] M. Jones, J. Bradley and N. Sakimura. “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, 2015. https://datatracker.ietf.org/doc/html/rfc7519

[6] N. Provos and D. Mazières. “A Future-Adaptable Password Scheme.” Proceedings of the USENIX Annual Technical Conference, 1999.

[7] express-validator. “express-validator Documentation.” https://express-validator.github.io/docs/

[8] Mozilla Developer Network. “MDN Web Docs: HTML, CSS and JavaScript.” https://developer.mozilla.org

[9] Postman, Inc. “Postman Learning Center.” https://learning.postman.com

[10] S. Russell and P. Norvig. Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020 (chapter on constraint satisfaction problems).

[11] OWASP Foundation. “OWASP Top 10 Web Application Security Risks.” https://owasp.org/www-project-top-ten/

[12] I. Sommerville. Software Engineering, 10th ed. Pearson, 2016.

[13] R. S. Pressman and B. R. Maxim. Software Engineering: A Practitioner’s Approach, 8th ed. McGraw-Hill Education, 2014.